1. Learning Algorithms (Optimization Behavior)

Answer:

Gradient Descent minimizes the loss function by updating model parameters using the learning rate (η).

Update Equation:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

where:

- θ = model parameters
- η = learning rate
- ∇J(θ) = gradient of the loss function

Effect of a High Initial Learning Rate

A high initial learning rate allows the optimizer to take large steps toward the minimum.

Advantages:

- Faster movement during early training.
- Escapes shallow local regions and plateaus.
- Reduces training time.

Effect of Learning Rate Decay

As training progresses, the learning rate is gradually reduced.

Benefits:

- Smaller parameter updates.
- Stable convergence near the optimum.
- Reduces oscillations around the minimum.
- Improves final accuracy.

When It Outperforms a Constant Learning Rate

This strategy performs better when:

- The loss function is smooth or convex.
- The optimizer is far from the optimum initially.
- Fine-tuning is required near convergence.

- Large datasets require efficient optimization.

Possible Risks

- If the initial learning rate is too high, the optimizer may overshoot the minimum.
- Training may become unstable or diverge.
- If the learning rate decreases too quickly, learning may stop before reaching the optimum.

Conclusion

A high initial learning rate followed by gradual decay combines **fast initial convergence** with **stable final optimization**, making it more effective than a constant learning rate in many machine learning problems.

2. Maximum Likelihood Estimation (MLE)

Answer

Assume

$$X_i \sim N(\mu, \sigma^2)$$

where:

- μ = unknown mean
- σ² = known variance

Likelihood Function

$$L(\mu) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(\quad -\frac{(x_i - \mu)^2}{2\sigma^2} \right)$$

Taking the logarithm,

$$\log L(\mu) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2$$

Differentiate with respect to μ:

$$\frac{d}{d\mu}\log L(\mu) = \frac{1}{\sigma^2}\sum(x_i - \mu)$$

Set equal to zero:

$$\sum(x_i - \mu) = 0$$

Therefore,

$$\boxed{\hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} x_i}$$

Hence, the **MLE of μ is the sample mean**.

Effect of Outliers

- The sample mean changes significantly when extreme values are present.
- Even one large outlier can shift the estimate considerably.
- Therefore, MLE is **sensitive to outliers**.

Effect of Increasing Sample Size

As the number of samples increases:

- The estimate becomes more stable.
- Variance decreases.
- Accuracy improves.
- The estimator converges to the true mean (Consistency).

Implication About Robustness

MLE based on the Gaussian distribution is:

- Efficient for clean data.
- Consistent with large samples.
- **Not robust** to outliers because it depends on the arithmetic mean.

3. Building a Machine Learning Algorithm (Model Selection)

Answer

Challenges

The dataset contains:

- **10,000 features**
- **500 training samples**

This creates several problems:

- High dimensionality
- Overfitting
- Increased computational cost
- Irrelevant and redundant features

Proposed Pipeline

Step 1: Data Preprocessing

- Handle missing values.
- Normalize or standardize features.

Reason:
Ensures all features are on the same scale.

Step 2: Feature Selection

Use:

- Mutual Information
- Chi-square test
- LASSO

Reason:
Removes irrelevant features and reduces dimensionality.

Step 3: Dimensionality Reduction

Apply PCA.

Reason:
Transforms thousands of correlated features into a smaller number of informative components.

Step 4: Regularization

Use:

- L1 Regularization (LASSO)
- L2 Regularization (Ridge)

Reason:
Prevents large weights and reduces overfitting.

Step 5: Model Training

Train using:

- Logistic Regression
- Linear SVM
- Small Neural Network

Step 6: Cross Validation

Use k-fold cross-validation.

Reason:
Provides reliable performance estimation and avoids overfitting.

How It Reduces Overfitting

- Feature selection removes noisy variables.
- PCA lowers dimensionality.
- Regularization limits model complexity.
- Cross-validation ensures good generalization.

4. Neural Networks (Multilayer Perceptron - MLP)

Answer

An MLP consists of:

- Input layer
- Hidden layer
- Output layer

The hidden layer usually uses sigmoid activation:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Why More Hidden Neurons Increase Representational Power

Each hidden neuron learns different nonlinear patterns.

Adding more neurons enables the network to:

- Learn more complex decision boundaries.
- Represent highly nonlinear relationships.
- Approximate almost any continuous function (Universal Approximation Theorem).

Thus, the network can model increasingly complex data.

Why More Neurons Do Not Always Improve Performance

Adding too many neurons causes:

- More trainable parameters.
- Increased model complexity.
- Higher risk of memorizing training data.

This results in **overfitting**, where:

- Training accuracy becomes very high.
- Test accuracy decreases.

Generalization

A good model should perform well on unseen data.

Too many hidden neurons reduce generalization because the model learns noise instead of meaningful patterns.

Conclusion

Increasing hidden neurons improves learning capacity but may reduce generalization. Therefore, the number of neurons should be carefully chosen to balance **model complexity** and **prediction performance**.

5. Backpropagation, SGD & Curse of Dimensionality

Answer

The **curse of dimensionality** refers to problems that arise when the number of features becomes very large.

Impact on Gradient Updates

Backpropagation computes gradients for every parameter.

In high-dimensional space:

- More parameters must be updated.
- Gradients become noisy.
- Vanishing or exploding gradients may occur.
- Training becomes less stable.

Impact on SGD Convergence

SGD updates weights using mini-batches.

With many dimensions:

- Optimization becomes more difficult.
- Loss surface becomes more complex.
- More iterations are required.
- Convergence becomes slower.

Impact on Generalization

High-dimensional data increases:

- Model complexity.
- Risk of overfitting.
- Poor prediction on unseen data.

This occurs because the model learns noise instead of useful patterns.

Techniques to Overcome These Problems

1. Dimensionality Reduction (PCA)

- Removes redundant features.

- Reduces computation.
- Speeds up convergence.
- Improves generalization.

2. Regularization (L1/L2, Dropout)

- Penalizes overly large weights.
- Prevents overfitting.
- Improves model robustness.

3. Batch Normalization (Optional)

- Normalizes activations.
- Stabilizes gradient flow.
- Accelerates convergence.

4. Adaptive Optimizers (Adam/RMSProp) (Optional)

- Adjust learning rates automatically.
- Improve optimization efficiency.
- Converge faster than standard SGD.

Conclusion

The curse of dimensionality slows learning, increases computational cost, and reduces generalization. Applying **dimensionality reduction**, **regularization**, **batch normalization**, and **adaptive optimizers** helps achieve faster convergence and better model performance.